SeatWise: A Constraint-Optimised, Reproducible Examination Seating Management System

A.SANJAY
Roll No. 23K91A6702
Department of Computer Science and Engineering (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

CH.NAVEEN
Roll No. 23K91A6743
Department of Computer Science and Engineering (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

D.KOTESH
Roll No. 24K95A6705
Department of Computer Science and Engineering (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

A.AKSHITH
Roll No. [Roll number to be confirmed]
Department of Computer Science and Engineering (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

[GUIDE NAME]
[DESIGNATION], Project Guide
Department of Computer Science and Engineering (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

***Abstract*—**Examination seating is still prepared by hand or in spreadsheets, which is slow, error-prone and predictable: sequential plans seat candidates who write the same paper side by side and invite malpractice. This paper presents SeatWise, a digital examination seating management system built around a constraint-optimisation engine. Hall seats are coloured so that seats of one colour are never neighbours. A first CP-SAT stage allocates candidates to halls and confines each paper to one colour class, which makes same-paper neighbours impossible by construction; a second stage lays out the seats of each hall with roll-number spacing, accessible seats and department mixing under seeded random objectives. Plans are checked by an independent validator and can be reproduced from a recorded seed. A web application adds validated import, live-checked seat editing, printable charts, QR seat slips, attendance and analytics. On eight seeded synthetic scenarios, all 24 plans met every hard rule with no same-paper neighbours, against 1,253 for roll-order seating at 600 candidates, and 600 candidates were seated in 2.4 s. The evidence is synthetic, and no user study has yet been carried out.

***Index Terms*—**constraint programming, CP-SAT, examination seating, graph colouring, malpractice prevention, reproducibility, web application.

I. INTRODUCTION

Every examination cycle begins with deciding who sits where. For each sitting the examination cell must turn candidate lists, course registrations, hall layouts and a timetable into a seating plan that invigilators can follow and candidates can find. A good plan does more than fill rooms: it limits malpractice by keeping apart candidates who write the same paper.

In practice most institutions still prepare seating by hand or in spreadsheets [1]. The base paper describes committees counting room capacities and assigning roll numbers one by one, which takes days for thousands of students, produces duplicated or missing seats and has to be redone after any change [1]. Roll-order seating is also predictable. In our own benchmark sitting of 600 candidates writing eight papers it leaves 1,253 pairs of neighbours who write the same paper; interleaving papers in a repeating cycle still leaves 281, and a random shuffle leaves 265. Software that automates the task mostly applies a fixed sort or interleaving pattern [1], [4].

The gap is a system that guarantees its spacing rules for any number of papers, chooses among valid plans in a way that is unpredictable yet reproducible, reports measured evidence, and supports the whole examination day. This paper presents SeatWise, an AI-enabled digital examination seating management system. Here "AI" means combinatorial optimisation with controlled randomness; no machine-learning model is trained. The contributions are:

1) a colour-class construction that makes same-paper neighbours impossible by construction, solved as a two-stage CP-SAT model with seeded random objectives;
2) reproducibility and independent validation: every plan stores its seed and three SHA-256 fingerprints, and a solver-free validator checks it before it is stored;
3) a complete web application with validated import, live and explained seat editing, PDF and Excel outputs, QR slips, public lookup, tablet attendance, analytics and an audit trail;
4) a seeded benchmark against three manual baselines, reported together with its limits.

Section II reviews related work, Section III describes the method, Section IV reports the results and Section V concludes.

A. System Hardware

All measurements were taken on one desktop with a 12-thread Intel processor (Intel64 Family 6 Model 151) running Windows 11 and Python 3.11.9. The amount of memory was not recorded and no other hardware was tested. Tablets and phones are optional clients of the web interface.

B. Software Requirements

The back end is Python 3.11 with FastAPI 0.141 [17], Pydantic 2, SQLAlchemy 2.0 [18] over SQLite, PyJWT and bcrypt, NumPy 2.4, pandas 3.0, openpyxl, ReportLab 5.0 and qrcode. The solver is Google OR-Tools 9.15.6755 (CP-SAT) [14]. The front end is React 18.3 [19] with TypeScript 5.8, Vite 6.3, Tailwind CSS, TanStack Query, Recharts and dnd-kit. Tests use pytest 9.1. Everything runs offline on Windows 10 or 11 with no API key.

II. LITERATURE SURVEY

A. Base Paper

Yukth et al. [1] present ExamSeater, a Flask and SQLite web application in a Model-View-Controller arrangement,

using SQLAlchemy, pandas, PyPDF2 and ReportLab. Student lists are uploaded as CSV or PDF. Its allocation is a *zipper* algorithm: two subject lists are sorted (by name, or by branch when a branch constraint is on) and interleaved with itertools.zip_longest so that subjects alternate, and blocked seats are skipped. The system has role-based login with PBKDF2-hashed passwords, management of rooms, exams and staff, an on-screen grid, a PDF seating chart, a history of arrangements and a public seat search by USN or name.

The paper reports no dataset and no measured result; its only quantitative statement is a requirement that a batch of 100 students be processed in under 3 s. Interleaving covers two subjects in a line, so with three or more papers in a sitting, or with diagonal neighbours, it does not guarantee that neighbours differ. The pattern is fixed and therefore predictable, and there is no roll-number rule, no accessible-seat handling, no attendance, no audit trail and no reproducibility mechanism. The authors list a mobile application, e-mail and SMS, face-recognition attendance, cloud hosting and drag-and-drop swapping as future work. SeatWise addresses the missing guarantee, the predictability and the missing measurement, and it already provides drag-and-drop editing and attendance.

B. Related Work

Web-based systems. Jitha and Fathima [7] propose a central portal with CSV upload and an online seat table; the base paper notes that it does not interleave subjects. An AI-powered hall-allocation system [10] uses a deterministic constraint-based method in Flask and MySQL and is described as weak at splitting students over partly used rooms. Patel [8] combines biometric attendance with QR seat lookup at a hardware and connectivity cost that the base paper judges too high for typical colleges. Works [7], [8] and [10] are described as the base paper describes them. Three further systems from 2025 and 2026 [2], [3], [5] show that the problem is current; their abstracts could not be retrieved, so we do not assess them.

Randomisation. An allocation based on a linear congruential generator [9] shuffles seats completely; as the base paper notes, this confuses invigilators and gives no guarantee about neighbours.

Colouring and metaheuristics. Adetona and Akintoye [4] seat students with a greedy graph-colouring algorithm over a connection matrix so that students of the same course are not adjacent, and send SMS reminders; their abstract reports testing with a selected group of university students, and we read only the abstract. Kashyap et al. [6] use a genetic multi-objective algorithm over subject similarity, physical distance and seat visibility, which handles soft objectives but cannot guarantee hard rules. The classical bound for colouring in timetabling is due to Welsh and Powell [11].

Timetabling and constraint programming. Qu et al. [12] survey examination timetabling and separate hard constraints, which must hold, from soft ones, which are minimised. Constraint programming [13] supplies exact solvers, and CP-SAT [14] offers Boolean models, objectives, hints and a deterministic limit, which suits a seating engine that must be reproducible. TABLE I compares the works reviewed above.

TABLE I
COMPARISON OF RELATED WORK

Work	Approach	Data	Reported result	Limitation
[1]	Zipper interleave, Flask	Institution lists	None; 100 students < 3 s target	Two subjects; fixed
[4]	Greedy colouring, SMS	University group	Limited test (abstract)	One rule family
[6]	Genetic search	Not reviewed	Not reviewed	No hard guarantee
[7]	Portal, CSV, seat table	CSV lists	Per [1]	No interleave
[8]	Biometric + QR lookup	University	Per [1]	Costly; needs internet
[9]	LCG shuffle	Student lists	Per [1]	Confuses invigilators
[10]	Deterministic constraint rules	Not reviewed	Per [1]	Room overflow
[2], [3], [5]	Seating systems (titles)	Not retrieved	Not assessed	-

C. Research Gap

No reviewed system guarantees that no neighbour, diagonals included, writes the same paper for any number of papers, enforces roll-number spacing and accessible seats together, offers randomness that respects the rules, reports measured violations and solve times, or joins import, optimisation, reproducibility, live editing, attendance and audit in one application. SeatWise is designed to close these gaps.

III. METHODOLOGY

A. Overall Pipeline

The method splits seating into three checkable problems. *Data preparation* parses and validates the uploaded files. *Allocation* (Stage A) decides which candidates go to which hall and which colour class of the hall each paper uses. *Layout* (Stage B) decides the exact seat of every candidate inside its paper's class. An independent validator then re-checks the plan, and the plan is stored with its seed and fingerprints. Fig. 1 shows the architecture and Fig. 2 the control flow of the engine.

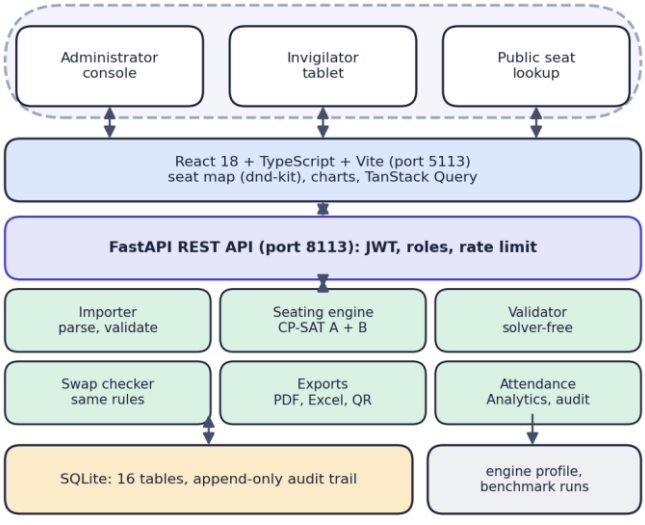


Fig. 1. System architecture of SeatWise.

B. Data and its Preparation

SeatWise needs no external dataset. Its sample data is generated by a seeded script (seed 2026) and is synthetic: 49 courses in 7 departments, 908 candidates with 2,724 registrations, 14 halls with 871 usable seats, and 6 sittings of 397 to 511 candidates and 7 to 9 papers; about 1% of candidates (10) need an accessible seat, and two courses share one question paper. No real person or record is involved. The benchmark scenarios are generated in memory from seeds, with about 1% of candidates needing an accessible seat and a target hall fill of 80%. Nothing is trained, so there is no training or test split.

Uploads (Excel or CSV for courses, halls, candidates and the timetable) are validated row by row before anything is saved: required columns, duplicates, unknown departments or courses, seat labels outside a layout, aisle positions, time order, split paper groups and timetable clashes. The sample workbook validates 1,020 rows with 0 errors and 10 warnings (halls listing no accessible seats receive two front-row seats by default). Accepted rows become plain engine records: a candidate has a paper key (the paper group when courses share a question paper), a department and an accessibility flag, and a hall becomes a seat grid with blocked seats, accessible seats and aisles. Candidates are grouped by (paper, department, accessible need), and these groups are what Stage A allocates.

C. Algorithm and Rationale

Sorting by roll number and zipper or round-robin interleaving give no guarantee for diagonals or several papers; a random shuffle gives none at all; greedy colouring and genetic search are heuristics that may end with violations. Constraint programming returns either a plan that satisfies every hard constraint or a proof that none exists, which lets SeatWise explain impossible requests, and adding a rule means adding a constraint. A single model over all seats of all halls would be large, so the problem is split into an allocation stage and a per-hall layout stage, which keeps the models small, lets halls be solved in parallel and enables the colour-class construction below.

D. Formulation

Let C be the candidates of a sitting, H the halls, p(c), d(c), r(c) and a(c) the paper, department, roll number and accessibility flag of candidate c, and S(h) the usable seats of hall h. Two seats are neighbours if they are adjacent in a row, a column or (in 8-neighbour mode) a diagonal with no aisle between them; blocked seats and aisles only remove edges. The seat in row i and column j receives the colour class

$$\kappa(i, j) = 2(i \bmod 2) + (j \bmod 2) \quad [4 \text{ classes}] \qquad (1)$$

(for 4-neighbour adjacency, $\kappa = (i + j) \bmod 2$, two classes). *Lemma:* two different seats of one class are never neighbours, because their row and column differences are both even and not both zero, so one is at least 2; in 4-neighbour mode neighbours differ in i + j by exactly 1.

Stage A. With N(g) the size of group g, integer n(h, g) the number of candidates of g in hall h, Boolean z(h, p, k) meaning that paper p uses class k of hall h, and integer m(h, p, k) its share there:

$$\Sigma_h\, n(h, g) = N(g); \quad \Sigma_g\, n(h, g) \le cap(h) \qquad (2)$$

$$\Sigma_k\, z(h, p, k) \le 1; \quad \Sigma_k\, m(h, p, k) = \Sigma_{g \in p}\, n(h, g) \qquad (3)$$

$$\Sigma_p\, m(h, p, k) \le |S(h, k)|; \quad m \le |S(h, k)|\, z \qquad (4)$$

The first pass minimises, in order, the peak fill of any hall and the spread of papers and departments; the second pass keeps that optimum and maximises a weighted sum of the z variables with seeded random integer weights, so the classes and halls a paper receives cannot be predicted. The compact strategy first picks the fewest, largest halls that a greedy best-fit packing can fill.

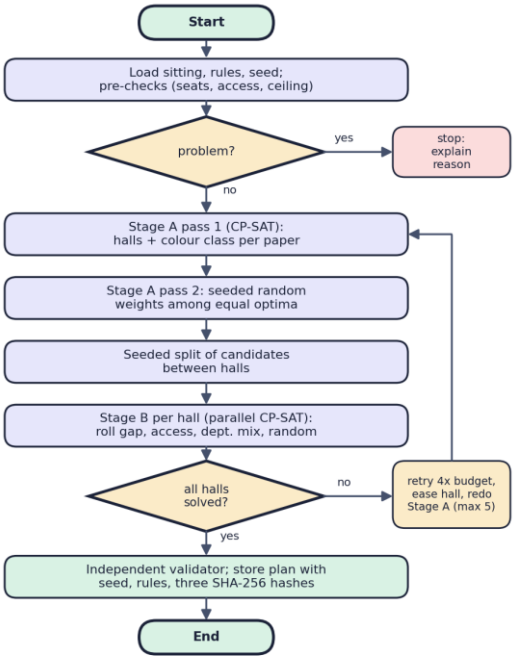


Fig. 2. Control flow of the seating engine.

Stage B. For each hall, a Boolean x(c, s) is 1 if candidate c sits on seat s of its class (only accessible seats for candidates with a(c) true):

$$\Sigma_s\, x(c, s) = 1; \quad \Sigma_c\, x(c, s) \le 1 \qquad (5)$$

$$x(i, s) + \Sigma_{t \in N(s)}\, x(j, t) \le 1 \qquad (6)$$

$$\min \; W \Sigma\, \pi - \Sigma\, w(c, s)\, x(c, s) \qquad (7)$$

Constraint (6) is posted for every pair i, j with the same roll prefix, $|r(i) - r(j)|$ below the gap and different classes (pairs in one class are never neighbours). In (7), π is a penalty for two neighbours of one department, w(c, s) a seeded random integer in [0, 20) and $W = 20|C(h)| + 1$, so department mixing takes precedence and the random term picks among equal layouts.

Why the hard rules hold. Same-paper neighbours are impossible because all candidates of a paper in a hall sit in one class (Stage A) and seats of a class are never neighbours (Lemma); roll gaps hold across classes by (6) and within a class by the Lemma; accessible needs hold because the allowed seats are accessible; one seat per candidate holds by

(5). Independently of this argument, the validator re-checks all four rules and a failing plan is never stored.

E. Implementation

Sub-seeds are derived from the plan seed with SHA-256, candidates are processed in a fixed order, and every solver call uses one worker and a deterministic time budget (0.15 per hall, tuned; 0.3 for Stage A), so the same data, rules, seed and engine version give the same plan. A plan stores a fingerprint of its inputs, a hash of the solver result and a hash of the current seating; a *Verify* action re-runs the engine and compares them. Halls are solved in parallel threads; a hall that finds nothing in budget is retried at four times the budget, and otherwise its capacity is eased and Stage A is re-run, at most five times. Pre-checks turn impossible requests into plain-language reasons.

The FastAPI back end has 57 operations over 16 SQLite tables, JWT sessions [15] with bcrypt password hashes [16], two roles, rate-limited public endpoints that show published seats and a shortened name only, and an append-only audit trail. A swap checker applies the same neighbour and roll-gap functions to manual moves, and the React seat map colours every target green or red with the reason. ReportLab produces seating charts, invigilator sheets and QR slips, and openpyxl produces hall lists and attendance reports. The delivered code is about 5,800 lines of Python in the back end (1,000 in the engine) with 58 automated tests.

F. Seat Editing and Outputs

After generation the administrator reviews the plan on a seat map. Picking up a candidate calls a swap-check endpoint that evaluates every other seat with the same neighbour, roll-gap and accessibility functions as the engine and returns whether the move keeps every rule and, if not, the reasons; the map colours the seats green or red accordingly (Fig. 5). A legal move is applied, the seating hash is updated and the move is audited, so that Verify can later report how many manual moves separate the stored plan from the engine's result. Published plans feed the printed outputs, the public lookup, which returns published seats only, and a tablet register in which each hall's attendance is marked by tap or by scanning a seat slip and is locked on submission. A paper can occupy at most the largest colour class of each selected hall, about a quarter of the seats in 8-neighbour mode and half in 4-neighbour mode, and the pre-check compares each paper with the sum of these ceilings, so a request that cannot be met is refused with the paper and the number involved.

IV. RESULTS

A. Test Conditions

Because nothing is trained, accuracy, F1 and confusion matrices do not exist for SeatWise; the engine is evaluated by a benchmark of rule violations and speed. Run engine-benchmark-20260921 covers 8 seeded synthetic scenarios of 150 to 2,000 candidates (4 to 14 papers, 3 to 8 departments, 3 to 39 halls, 8 or 4 neighbours, roll gap 0, 5 or 10), four methods and three seeds (11 to 13), plus predictability over six seeds, on the desktop of Section I-A with a per-hall budget of 0.15. The baselines are sequential (roll order), round-robin (papers interleaved cyclically) and random shuffle. Every plan is checked by the independent validator.

B. Rule Compliance and Speed

All 24 SeatWise plans met every hard rule, with no same-paper neighbours, no roll-gap violations and no accessible seat missed; none of the 72 baseline runs met them (TABLE II, Fig. 3). Mean solve times were 1.25, 2.03, 2.36, 2.55 and 4.77 s for 150, 300, 600, 1,000 and 2,000 candidates; four-neighbour mode took 6.02 s at 600 candidates, the slowest run in the benchmark being 6.14 s. Tuning the per-hall budget over 0.1, 0.15, 0.25, 0.4 and 0.6 gave 3.72, 2.63, 3.56, 5.01 and 6.58 s for a whole sitting, with 127.3, 123.3, 119.3, 116.0 and 114.0 same-department pairs; 0.15 is the fastest budget within 10% of the best department mix.

TABLE II
MEASURED RESULTS, MEAN OF THREE SEEDS

Measure (600 candidates)	SeatWise	Seq.	Round-robin	Shuffle
Same-paper pairs	0	1,253	281	265
Roll-gap violations	0	453	198	23
Accessible seats missed	0	6	7	7
Same-department pairs	109	1,648	324	366
Solve time (s)	2.36	<0.01	<0.01	<0.01
Hard rules met (all runs)	100%	0%	0%	0%
Roll-to-seat correlation	0.065	1.000	0.213	0.034
Repeated neighbour pairs	1.0%	100%	100%	1.0%
Front-row bias (pp)	4.7	12.4	3.4	1.6

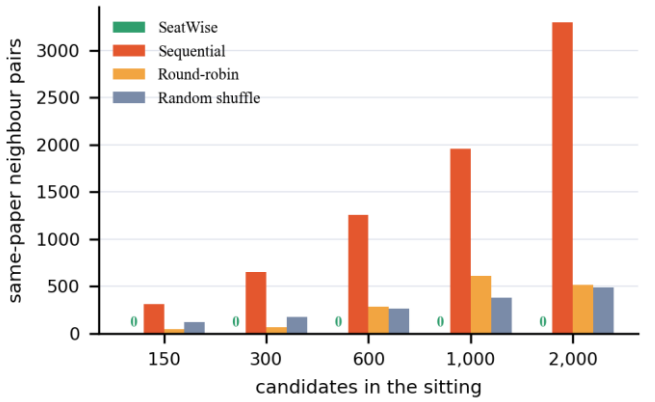


Fig. 3. Same-paper neighbour pairs by method and sitting size (SeatWise: 0 in every run).

C. Scaling and Variants

Across the five sizes SeatWise used 3, 5, 9, 12 and 28 halls and filled 71%, 82%, 96%, 96% and 97% of their seats, leaving 23, 7, 109, 190 and 597 same-department pairs, while sequential seating left 307, 648, 1,253, 1,954 and 3,299 same-paper pairs. The variants at 600 candidates behaved as expected. Four-neighbour adjacency left 12 same-department pairs against 109, at the cost of 6.02 s; a roll gap of 0 or 10 took 2.31 and 2.42 s and left 108 and 104 same-department pairs. With the gap rule on, the baselines violated it 453 (sequential), 198 (round-robin) and 23 (shuffle) times at a gap of 5, and 921, 252 and 48 times at a gap of 10; SeatWise never did.

D. Unpredictability

Over six seeds, SeatWise is as unpredictable as a pure random shuffle: the correlation between roll order and seat is

0.065 (sequential 1.000) and only 1.0% of neighbour pairs repeat when the plan is regenerated with another seed (sequential and round-robin 100%). Unlike the shuffle it keeps every rule. Its front-row bias, 4.7 percentage points against 1.6 for the shuffle, is the price of the colour-class construction: only two of the four classes include the front row.

E. Product-Level Results

The 58 backend tests passed on 15 October 2026. A scripted scenario on a fresh database imported the sample workbook, generated and published plans for three sittings (451, 457 and 451 candidates in 6, 7 and 7 halls, solved in 2.1, 1.8 and 2.2 s, with 0 same-paper pairs against 1,099, 1,175 and 1,105 for roll order), refused an illegal swap with a reason, downloaded all exports, looked up a seat and marked attendance, passing in 11.4 s. Verify reproduced a plan exactly in about 2.2 s. By module, the tests cover authentication and roles (8), the engine and its rules (14), import (9), plans, swaps and Verify (6), exports (7), attendance (5), public lookup (5) and analytics (4); they include same-seed reproduction, refusal of illegal swaps, impossible-request messages, role restrictions and rate limiting (HTTP 429 after 30 lookups a minute from one client). Local API response times were 2 to 11 ms (median) for data endpoints, 96 ms for the seating-chart PDF, 314 ms for the hall lists and 3.2 s for the 76-page QR-slip PDF. Fig. 4 shows the plan page of a published sitting and Fig. 5 the seat map refusing an illegal move.

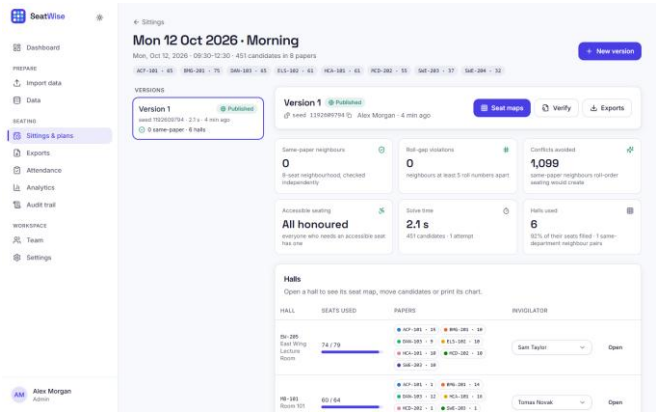


Fig. 4. Plan page: scorecard of a published sitting (0 same-paper neighbours, 1,099 conflicts avoided, 2.1 s).

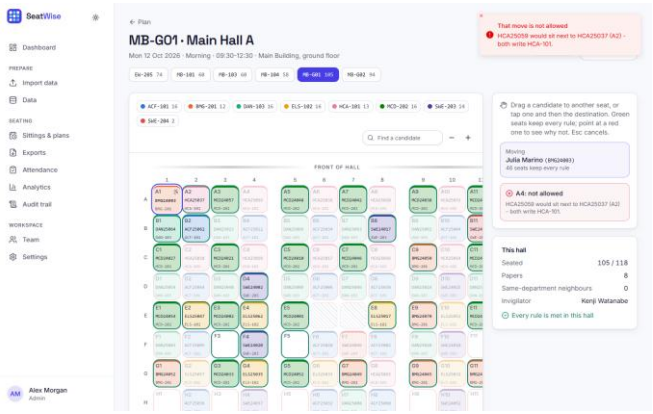


Fig. 5. Seat map: an illegal move is refused with the reason.

F. Comparison with the Base Paper

The comparison is qualitative, because the base paper [1] reports no dataset, no accuracy and no timing except a target of 100 students in under 3 s, and its system was not run on our data. The closest measurable proxy for its interleaving idea is our round-robin baseline, which is not a re-implementation of it: at 600 candidates it leaves 281 same-paper pairs and 198 roll-gap violations and repeats 100% of neighbour pairs, whereas SeatWise leaves none, and it also provides the live editing and attendance that [1] lists as future work. The datasets and conditions differ, so no numerical advantage over [1] is claimed.

G. Failure Cases and Limits

An impossible request is refused with reasons and nothing is stored: for two small halls and a 457-candidate sitting the reply was “457 candidates but only 72 usable seats in the selected halls. Select halls with at least 385 more seats,” followed by the papers that exceed the per-paper ceiling. A per-hall budget of 0.1 was slower overall (3.72 s) than 0.15 (2.63 s) because some halls ran out of time and were solved again with a larger budget. An import file with a duplicate roll number, an unknown department, an unknown course and an invalid flag failed with four row-level errors. Known limits: all data is synthetic; the benchmark was run once on one machine; no user-acceptance, load or penetration test was carried out; the front-row bias and the slower four-neighbour mode are real weaknesses; and a single paper larger than the colour-class capacity of the selected halls cannot be seated.

H. Discussion

Correctness here is structural rather than statistical: because a paper is confined to one colour class, zero same-paper neighbours follows from the construction and was confirmed by the independent validator in every run, rather than being an average over lucky plans. Speed is governed mainly by the deterministic per-hall budget and grows slowly with sitting size, so a whole sitting is planned in the time it takes to read the scorecard. What the results do not show is that real sittings behave like synthetic ones, or that examination staff will find the interface easier than their present tools; both need a pilot.

V. CONCLUSION

SeatWise automates examination seating with a two-stage CP-SAT engine that confines each paper to one colour class of a hall, so that same-paper neighbours are impossible by construction, while seeded random objectives, an independent validator and stored fingerprints make plans unpredictable, correct and reproducible. Against the project objectives, seats are allocated from uploaded data, capacity, subject and roll-number constraints hold, spacing is enforced and seating is randomised, charts and reports are exportable, lookup and attendance work, and plans take seconds: 24 of 24 benchmark plans met every hard rule and 600 candidates were seated in 2.4 s. The evidence is synthetic, from one machine and without a user study, so real-world performance is not yet established, and the front-row bias and the slower four-neighbour mode remain.

Future work: a pilot on a real anonymised sitting with feedback from examination staff; rotating colour classes to reduce the front-row bias and tuning four-neighbour mode; e-mail or SMS seat notifications and a mobile app; and a server database with HTTPS and a security review for multi-institution deployment.

REFERENCES

[1] P. Yukth, I. S. Mahmood, K. H. Shashank, S. G. Thejus and M. S. Darshini, "Exam Seating Arrangement System," International Journal of Research Publication and Reviews (IJRPR), vol. 6, no. 12, pp. 2093-2105, Dec. 2025.

[2] "Automated examination seating arrangement and hall allocation system," International Journal of AI, Electrical, Civil and Mechanical Engineering, 2026.

[3] "An intelligent machine learning-based system for automated examination hall seating arrangement in universities," in Proc. 2026 Int. Conf. AI-Driven Solutions for Sustainable Smart Cities: Challenges and Opportunities (ADSSSC), 2026.

[4] S. O. Adetona and F. A. Akintoye, "Design and implementation of an examination seating arrangement application to curb examination malpractice," ABUAD Journal of Engineering Research and Development (AJERD), vol. 6, no. 2, pp. 143-156, Dec. 2023.

[5] "Automated seating arrangement for conducting examination," in Proc. 2025 Int. Conf. Multi-Agent Systems for Collaborative Intelligence (ICMSCI), 2025.

[6] Kashyap et al., "Exam seating allocation to prevent malpractice using genetic multi-optimization algorithm," Springer, 2021, doi: 10.1007/978-981-16-0419-5_11.

[7] K. Jitha and A. Fathima, "Web application based exam hall seating management system," in Proc. IEEE Int. Conf. Applied Artificial Intelligence and Computing (ICAAIC), 2022 (as described in [1]).

[8] M. Patel, "Smart seating system for universities," in Proc. IEEE Int. Conf. Computing, Communication and Automation, 2021 (as described in [1]).

[9] "Automated students examination seat allocation using linear congruential generator," International Journal of Computer Trends and Technology (IJCTT), 2021 (as described in [1]).

[10] "AI powered automated exam hall allocation and management system," International Journal of Science and Research (IJSART), 2025 (as described in [1]).

[11] D. J. A. Welsh and M. B. Powell, "An upper bound for the chromatic number of a graph and its application to timetabling problems," The Computer Journal, vol. 10, no. 1, pp. 85-86, 1967.

[12] R. Qu, E. K. Burke, B. McCollum, L. T. G. Merlot and S. Y. Lee, "A survey of search methodologies and automated system development for examination timetabling," Journal of Scheduling, vol. 12, no. 1, pp. 55-89, 2009.

[13] F. Rossi, P. van Beek and T. Walsh, Eds., Handbook of Constraint Programming. Amsterdam, The Netherlands: Elsevier, 2006.

[14] L. Perron and V. Furnon, "OR-Tools," Google. [Online]. Available: https://developers.google.com/optimization

[15] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015.

[16] N. Provos and D. Mazieres, "A future-adaptable password scheme," in Proc. USENIX Annual Technical Conference, 1999.

[17] S. Ramirez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com

[18] M. Bayer, "SQLAlchemy," in The Architecture of Open Source Applications, vol. 2, A. Brown and G. Wilson, Eds., 2012.

[19] Meta Platforms, "React documentation." [Online]. Available: https://react.dev